

1. JYOTHI, V.; ADDEPALLI, S. K.; KARRI, R. Deep Packet Field Extraction Engine (DPFEE): a pre-processor for network intrusion detection and denial-of-service detection systems. *Proc. 33rd IEEE ICCD*, 2015, p. 266-272.

Extração de campos de protocolo em hardware como pré-processador em linha com o NIDS, via múltiplos micro-engines paralelos. Trata SIP e HTTP em Virtex-7 VC707, validado com capturas do Wireshark e tráfego VoIP real; atinge 408,5 Gbps. Granularidade por pacote, sem estado.

Artigo: <https://patents.google.com/patent/US10735438B2/en>

2. PHAM-QUOC, C.; TRAN, N. T. H.; THINH, T. N. FPGA/AI-powered architecture for anomaly network intrusion detection systems. *Electronics*, v. 12, n. 3, art. 668, 2023.

NIDS completo em FPGA com módulo FEN de extração e normalização de características alimentando uma rede neural. Verilog-HDL, Vivado, Virtex-7 (NetFPGA-SUME): 21,39% dos LUTs a 206,7 MHz, validado por replay de PCAP com tcpreplay. Usa features de fluxo (CIC-IDS2017 via CICFlowMeter).

Artigo: <https://www.mdpi.com/2079-9292/12/3/668/pdf>

3. LU, Y. et al. Octopus: a heterogeneous in-network computing accelerator enabling deep learning for network. *arXiv:2308.11312*, 2023.

Implementa em Verilog um *flow tracker* com tabela de estado de 8k fluxos. Publica o custo isolado do extrator: 9051 LUTs (1,4%) e 21,5 BRAMs (1,0%) a 125 MHz em Xilinx xcku-115. Única referência com números de recursos de um extrator por fluxo em RTL — baseline de dimensionamento.

Artigo: <https://arxiv.org/pdf/2308.11312>

4. HAN, Z.; BRIASCO-STEWART, N.; ZINK, M.; LEESER, M. Extracting TCP/IP headers at high speed for the anonymized network traffic graph challenge. *IEEE HPEC*, 2024.

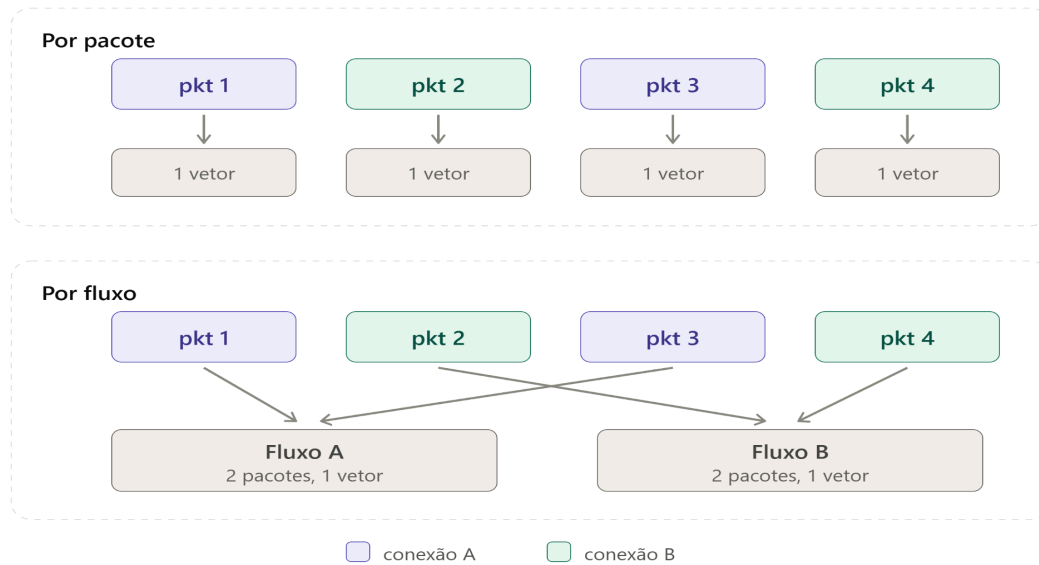
Extração de cabeçalhos Ethernet/IPv4/TCP em SmartNIC, combinando parser P4 com blocos com estado em HLS (via *externs*, já que P4 é stateless). Cerca de 95 Gbps. Contraponto metodológico que justifica a opção por RTL manual.

Artigo: <https://arxiv.org/pdf/2409.07374>

Anotações

Granularidade de extração:

A granularidade define o que conta como uma amostra para o modelo de ML. Na granularidade de pacote, cada pacote gera um vetor de características e o módulo é *stateless*; na granularidade de fluxo, os pacotes são agrupados pela 5-tupla e um único vetor é emitido por conexão, o que exige uma arquitetura *stateful* com tabela de fluxos.



Por pacote

Cada pacote que entra sai como um vetor. O módulo é **sem memória**: lê os cabeçalhos, recorta os campos, emite. O pacote seguinte não sabe nada do anterior.

As features possíveis são só o que está escrito no próprio pacote:

src_ip, dst_ip, src_port, dst_port, protocol, ttl, tos, ip_flags, tcp_flags, window_size, total_length, header_length, payload_length

Por fluxo

O módulo agrupa os pacotes que pertencem à mesma conexão (a 5-tupla) e só emite um vetor **quando a conexão termina**. Ele precisa guardar estado enquanto isso não acontece.

Isso libera uma classe inteira de features que simplesmente não existem no nível do pacote:

duration, total_packets, total_bytes, mean/std/min/max do tamanho de pacote,

mean/std/min/max do intervalo entre chegadas (IAT), contagem de cada flag TCP no fluxo,

razão fwd/bwd de pacotes e bytes, bytes por segundo, pacotes por segundo.